Lab 1 report

Olena Azarova, Ihor Ivanyshyn

March 16, 2026

Task 1

1 Phoneme recognition for TIMIT dataset via SSL models.

The core of this research lies in a comparative analysis of two state-of-the-art self-supervised learning (SSL) architectures for speech: Wav2Vec and Data2Vec. While both models share a similar "backbone" (a convolutional feature encoder followed by a Transformer-based context network), they represent two distinct philosophies in pre-training:

Wav2Vec 2.0: Utilizes a contrastive task, where the model learns to distinguish a true quantized latent representation from a set of distractors.

Data2Vec-Audio: Utilizes a regression task, where the model predicts the latent representations of the full input data based on a masked view, without using quantization or contrastive distractors.

1.1 Data preprocessing

The dataset consists of 6,300 utterances spoken by 630 speakers from eight major dialects of American English. For this study, the data is partitioned into:

Train: Used for training the custom decoding heads.

Validation: Used for hyperparameter tuning and early stopping.

Test: A held-out set used for the final performance report.

1.2 Phoneme mapping

Raw TIMIT transcriptions include 61 phonetic symbols. However, many of these are variations of the same phoneme (e.g., different types of closures or silences). To follow standard practice, we implement a mapping that reduces the 61 labels into 39 broad phoneme classes.

The map looks like:

```
phon61_map39 = {
    'iy':'iy',  'ih':'ih',  'eh':'eh',  'ae':'ae',   'ix':'ih',  'ax':'ah',  '
        ah':'ah',   'uw':'uw',
    'ux':'uw',  'uh':'uh',  'ao':'aa',  'aa':'aa',   'ey':'ey',  'ay':'ay',  '
        oy':'oy',   'aw':'aw',
    'ow':'ow',  'l':'l',    'el':'l',   'r':'r',     'y':'y',    'w':'w',    '
        er':'er',   'axr':'er',
    'm':'m',    'em':'m',   'n':'n',    'nx':'n',    'en':'n',   'ng':'ng',  '
        eng':'ng',  'ch':'ch',
    'jh':'jh',  'dh':'dh',  'b':'b',    'd':'d',     'dx':'dx',  'g':'g',    '
        p':'p',     't':'t',
    'k':'k',    'z':'z',    'zh':'sh',  'v':'v',     'f':'f',    'th':'th',  '
        s':'s',     'sh':'sh',
    'hh':'hh',  'hv':'hh',  'pcl':'h#', 'tcl':'h#',  'kcl':'h#', 'qcl':'h#', '
        bcl':'h#',  'dcl':'h#',
    'gcl':'h#', 'h#':'h#',  '#h':'h#',  'pau':'h#',  'epi':'h#', 'nx':'n',   '
        ax-h':'ah', 'q':'h#'
}
```

A dictionary is applied to both the ground truth and the model's predictions during the evaluation phase to calculate the Phoneme Error Rate.

1.3 Audio processing

Before being fed into either the Wav2Vec or Data2Vec encoder, the raw waveforms undergo the following:

Resampling: All audio is standardized to 16,000 Hz.

Normalization: The input signal is normalized to have zero mean and unit variance.

Feature Extraction: The AutoFeatureExtractor corresponding to each model ensures that the temporal resolution matches the expected input of the convolutional layers.

1.4 Model Architectures

Both Wav2Vec 2.0 and Data2Vec-Audio are built on a similar high-level architecture for speech representation learning. In both models, raw audio is first passed through a convolutional feature encoder, which converts the waveform into a sequence of latent acoustic features. These features are then processed by a Transformer encoder, which models contextual information across time and produces progressively richer hidden representations at each layer.

For the extracted hidden states, the model outputs have the following shapes:

```
Layer 0:  torch.Size([1, 270, 768])
Layer 1:  torch.Size([1, 270, 768])
Layer 2:  torch.Size([1, 270, 768])
Layer 3:  torch.Size([1, 270, 768])
Layer 4:  torch.Size([1, 270, 768])
Layer 5:  torch.Size([1, 270, 768])
Layer 6:  torch.Size([1, 270, 768])
Layer 7:  torch.Size([1, 270, 768])
Layer 8:  torch.Size([1, 270, 768])
Layer 9:  torch.Size([1, 270, 768])
Layer 10: torch.Size([1, 270, 768])
Layer 11: torch.Size([1, 270, 768])
Layer 12: torch.Size([1, 270, 768])
```

We observe that 13 layers, each producing output tensors of identical dimensionality: (1, 270, 768). Here:

1 = batch size, so there is one audio sample in the input

270 = sequence length, meaning the audio was converted into 270 time-step representations, by default variable with audio length and also max = 270

768 = hidden dimension, so each time step is represented by a 768-dimensional feature vector

1.5 Heads

Each head is designed to process the output of a specific Transformer layer:

Linear: A standard single-layer projection directly to the phoneme vocabulary size.

MLP (Multi-Layer Perceptron): A series of linear layers with non-linear activations (GELU) to model more complex mappings.

BiLSTM: A Bidirectional Long Short-Term Memory network that captures sequential context from both past and future time steps in the hidden state.

Conv1D: A 1D convolutional network that focuses on local phonetic patterns by looking at neighboring audio frames.

LiteConformer: A lightweight implementation of the Conformer architecture. It combines Multi-Head Self-Attention (MHSA) for global context with 1D Convolutions for local features.

1.6 CTC loss

Also we used CTC loss, the general it is a loss function designed for sequence tasks where the input and target lengths are different, and where the exact alignment between them is unknown. This is very common in speech recognition and phoneme recognition: an audio signal may contain hundreds of frames, while the phoneme transcript is much shorter.

1.7 Connectionist Temporal Classification (CTC) Loss

CTC is a loss function designed for sequence tasks where the input and target lengths differ and the exact alignment between them is unknown. This is common in speech and phoneme recognition, where an audio signal may contain hundreds of frames while the transcript is much shorter.

CTC introduces a special blank token ϵ, representing "no output" at a time step. At each frame, the model predicts a probability distribution over the phoneme labels plus this blank symbol.

Let $\mathbf{x}$ be the input audio sequence and $\mathbf{y} = (y_1, \ldots, y_n)$ be the target phoneme sequence. After passing $\mathbf{x}$ through the encoder, the model produces probabilities for T time steps.

A CTC path $\pi = (\pi_1, \ldots, \pi_T)$ is a sequence of length T over the extended vocabulary (including ϵ). Assuming conditional independence, the probability of a single path is:

$$p(\pi|\mathbf{x}) = \prod_{t=1}^{T} p_t(\pi_t|\mathbf{x}) \tag{1}$$

where $p_t(\pi_t|\mathbf{x})$ is the predicted probability of symbol π_t at time t.

CTC defines a many-to-one collapse function, $\mathcal{B}$, which transforms a path into the final output sequence by merging consecutive repeated symbols and removing all blank symbols. The probability of the target sequence $\mathbf{y}$ is the sum of probabilities of all valid paths π that map to $\mathbf{y}$:

$$P(\mathbf{y}|\mathbf{x}) = \sum_{\pi \in \mathcal{B}^{-1}(\mathbf{y})} p(\pi|\mathbf{x}) \tag{2}$$

The loss function is defined as the negative log-likelihood of this probability:

$$\mathcal{L}_{\text{ctc}} = -\log P(\mathbf{y}|\mathbf{x}) \tag{3}$$

1.8 PER

The main metric to see performing is Phoneme error rate, it counts like:

$$PER = \frac{S + I + D}{N}$$

where S – Incorrect phonemes predicted in place of correct ones, I – extra phonemes added that were not in the reference, D – phonemes from the reference that were missed, N – total number of phonemes

1.9 Methodology

To understand where phonetic information is most densely represented, the project employs a frozen-feature probing methodology. This approach treats the pre-trained transformer as a static "sensor" while only training a small "decoder" on top of its outputs.

Rather than fine-tuning the entire model (which would adapt the backbone to the task), the encoder weights for both Wav2Vec and Data2Vec are set to eval() mode and their parameters are not updated during training. This ensures that the benchmarking results reflect the "inherent" knowledge captured during the models' original self-supervised pre-training.

The models consist of an initial convolutional feature projection followed by 12 Transformer layers. The pipeline follows these steps:

Hidden states are extracted from specific indices, typically ranging from layer 0 (feature projection) to 12 (final context layer).

To optimize compute, the extracted latent representations for the entire TIMIT dataset are saved as PyTorch tensors (.pt files) in a local cache directory (e.g.,

```
./cache/phoneme_features_data2vec/).
```

The system systematically loops through every combination of selected layer and classification head from the Model Zoo.

For each layer-head pair, a dedicated training run is performed. The performance is logged in a structured benchmark file, tracking metrics such as Training Loss, Validation PER, and Test PER across 10 epochs.

The *DataCollatorCTCWithPadding* class is a critical component of the implementation. It dynamically pads the audio features and the label sequences within each batch to the length of the longest element, ensuring that the model can process variable-length utterances from the TIMIT corpus efficiently.

1.10 Results

Wav2wec:

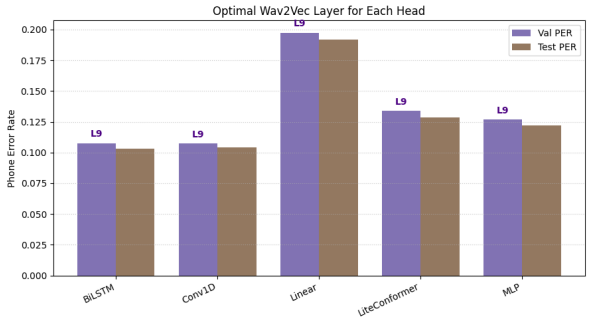

Figure 1: Each head best results Wav2Vec

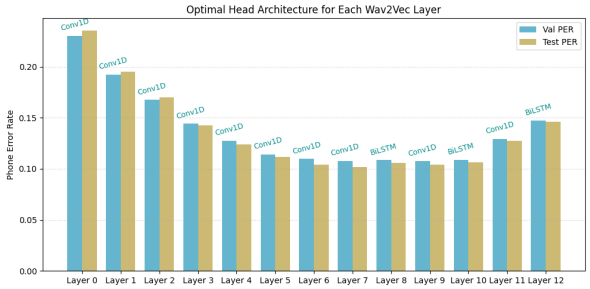

Figure 2: The best performance per layer Wav2Vec

Data2Vec:

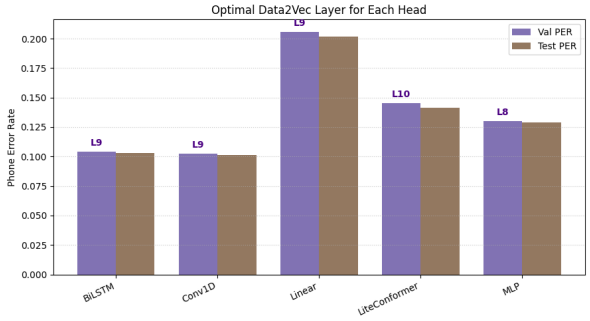

Figure 3: Each head best results Data2Vec

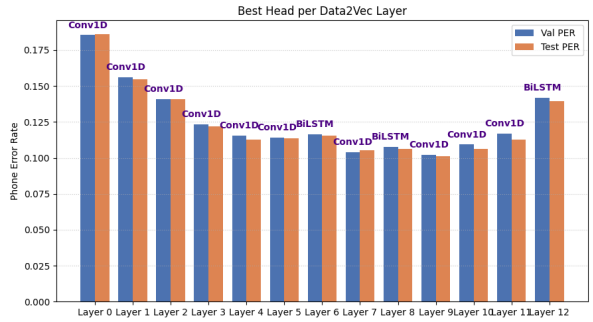

Figure 4: The best performance per layer Data2Vec

Also in jupyter notebook exist more detailed statistic about each head and layer.

The best performance on Wav2Vec has been shown by Conv1D on 7 layer – 0.1016 but on 9 epoch, if on full 10 epoch then on 9 layer, and for Data2Vec – also Conv1D bit on 9 layer – 0.1011.

Task 2

2 Filtering Algorithms

The goal of the proposed preprocessing procedure is to improve the acoustic quality of the TIMIT dataset using only classical digital signal processing methods. The main idea is to suppress low-frequency bias, narrow-band interference, out-of-band components, and stationary noise while preserving speech-relevant information.

The TIMIT dataset contains speech recordings sampled at 16 000 Hz therefore all audio files were processed at that target sampling rate.

2.1 Input signal preparation

Each waveform was first loaded from the disk and converted to a mono signal if necessary by averaging channels. Let the discrete-time input waveform be denoted by $x[n]$.

To quantify signal energy, the root mean square (RMS) value was defined as

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1} x[n]^2 + \varepsilon},$$

where N is the number of samples and ε is a small constant used for numerical stability.

2.2 DC offset removal

The first processing step removes the DC component of the signal:

$$y[n] = x[n] - \mu_x,$$

where μ_x is average signal value:

$$\mu_x = \frac{1}{N}\sum_{n=0}^{N-1} x[n].$$

This step centers the waveform around zero and eliminates constant bias, which may otherwise negatively affect further filtering stages.

2.3 Peak normalization

For amplitude stabilization we applied peak normalization. Let

$$m = \max_n |x[n]|.$$

If m is sufficiently large, the normalized signal is computed as

$$y[n] = \frac{x[n]}{m} \cdot p,$$

where $p = 0.98$ is the target peak amplitude. This keeps the signal within a safe dynamic range while avoiding clipping.

2.4 Pre-emphasis and de-emphasis

In order to amplify higher-frequency speech components before spectral denoising, we applied pre-emphasis:

$$y[0] = x[0], \qquad y[n] = x[n] - \alpha x[n-1], \quad n \geq 1,$$

where $\alpha = 0.97$.

After denoising, in order to restore the original spectral tilt we applied the corresponding de-emphasis filter:

$$y[0] = x[0], \qquad y[n] = x[n] + \alpha y[n-1], \quad n \geq 1.$$

Pre-emphasis is commonly used in speech processing because it makes weaker high-frequency components more prominent.

2.5 High-pass and low-pass Butterworth filtering

We used a combination of a high-pass and a low-pass Butterworth filter of the order 4 to isolate the speech band.

High-pass filter. A Butterworth high-pass filter with cutoff frequency was applied to suppress very low-frequency components such as slow drift, hum, and microphone handling noise.

$$f_{\mathrm{HP}} = 80 \text{ Hz}$$

Low-pass filter. A Butterworth low-pass filter with cutoff frequency was used to reduce high-frequency noise outside the main speech region.

$$f_{\mathrm{LP}} = 4000 \text{ Hz}$$

The resulting speech-band filtering stage can be written conceptually as

$$x_{\mathrm{bp}}[n] = H_{\mathrm{LP}}\big(H_{\mathrm{HP}}(x[n])\big),$$

This approximation to a bandpass filter is due to the fact that a significant portion of phonetically meaningful speech information is concentrated roughly in the range from 80 Hz to 4000 Hz.

2.6 Notch filtering of power-line interference

To suppress narrow-band interference related to electrical power-line noise, notch filters were applied at

$$50 \text{ Hz}, \quad 100 \text{ Hz}, \quad 150 \text{ Hz}.$$

These frequencies correspond to the fundamental power-line component and its lower harmonics. For each frequency f_0, we used an IIR notch filter with quality factor $Q = 30$ This filtering stage is especially useful when recordings may contain weak stationary tonal interference.

2.7 Adaptive Wiener filtering

After bandpass filtering, an adaptive Wiener filter was applied:

$$y = \mathrm{Wiener}(x).$$

In the implementation, the Wiener filter used a local window size of 29 samples. The purpose of this stage is to reduce local noise variance while preserving the main waveform structure. Wiener filtering is particularly attractive because it adapts to local statistics of the signal instead of applying a globally fixed attenuation.

2.8 Spectral gating denoising

The main denoising stage was implemented in the time-frequency domain using spectral gating.

2.8.1 STFT representation

First, the short-time Fourier transform (STFT) of the signal was computed using:

- FFT size: $N_{\mathrm{FFT}} = 512$,
- window length: 512,
- hop size: 128,
- Hann window.

For a frame index m and frequency bin k, the STFT can be written as

$$X(k,m) = \sum_n x[n]\, w[n - mR]\, e^{-j2\pi kn/N_{\mathrm{FFT}}},$$

where $w[\cdot]$ is the Hann window and R is the hop size.

The magnitude and phase were then extracted as

$$|X(k,m)|, \qquad \angle X(k,m).$$

2.8.2 Noise estimation

Noise statistics were estimated from the beginning and the end of each recording. Specifically, 0.20 seconds from the start and 0.20 seconds from the end were used as a rough estimate of background noise. Let the resulting noise-only signal be denoted by x_{noise}.

From its STFT magnitude, the mean and standard deviation were computed for each frequency bin:

$$\mu_k = \mathrm{mean}\big(|X_{\mathrm{noise}}(k,m)|\big), \qquad \sigma_k = \mathrm{std}\big(|X_{\mathrm{noise}}(k,m)|\big).$$

The threshold was then defined as

$$T_k = \mu_k + \lambda\sigma_k,$$

where

$$\lambda = 1.5.$$

2.8.3 Soft masking

Given the STFT magnitude $|X(k,m)|$ of the input signal, a binary decision was made:

$$M(k,m) = \begin{cases} 1, & |X(k,m)| \geq T_k, \\ g, & |X(k,m)| < T_k, \end{cases}$$

where

$$g = 0.15$$

is the floor gain.

Thus, coefficients below the threshold were not removed completely; instead, they were attenuated. This soft attenuation helps reduce musical-noise artifacts. The mask was additionally smoothed by a median filter of size $(5, 3)$ in the time-frequency plane.

The output magnitude was obtained as

$$|\hat{X}(k,m)| = M(k,m)\,|X(k,m)|.$$

Finally, the inverse STFT was applied using the preserved phase:

$$\hat{x}[n] = \mathrm{ISTFT}\left(|\hat{X}(k,m)|e^{j\angle X(k,m)}\right).$$

If necessary, the reconstructed signal was padded or truncated to match the original waveform length.

2.9 RMS normalization

After denoising, RMS normalization was used to bring all processed signals to a comparable loudness level. If the target level in linear scale is

$$r_{\text{target}} = 10^{L/20},$$

where $L = -22$ dB, and the current RMS value is r_{current}, then the gain factor is

$$g = \frac{r_{\text{target}}}{r_{\text{current}}}.$$

The normalized signal is

$$y[n] = g\,x[n].$$

In order to ensure a safe maximum amplitude we applied peak normalization after RMS normalization again.

3 Ablation pipelines

To study the contribution of the individual components, several simplified pipelines were also defined:

- **Base**: DC removal + RMS normalization + peak normalization.
- **Bandpass**: DC removal + speech band filtering + RMS normalization + peak normalization.
- **Bandpass_Wiener**: DC removal + speech band filtering + Wiener filtering + RMS normalization + peak normalization.
- **Full**: the complete enhancement pipeline described above.

These variants were later used in the ablation study to determine which processing stages contribute the most to the final speech quality and recognition performance.

3.1 Objective evaluation metrics

In addition to the phoneme error rate several other signal-level objective measures were computed.

Spectral centroid. The spectral centroid was used to measure the average frequency balance of the signal:

$$C = \frac{\sum_f f\,S(f)}{\sum_f S(f) + \varepsilon},$$

where $S(f)$ is the spectrogram magnitude or power. Lower values generally indicate stronger suppression of high-frequency energy.

Log spectral distance (LSD). To quantify spectral distortion between the original and processed waveforms, the log spectral distance was used:

$$\mathrm{LSD}(x, y) = \sqrt{\frac{1}{KM} \sum_{k,m} \left(\log(|X(k,m)|^2 + \varepsilon) - \log(|Y(k,m)|^2 + \varepsilon)\right)^2}.$$

Signal-to-noise ratio (SNR) proxy. A proxy signal-to-noise ratio was computed as

$$\mathrm{SNR}(x, y) = 10 \log_{10} \left(\frac{E[x^2]}{E[(x-y)^2] + \varepsilon} \right),$$

where x is the original signal and y is the processed one. In this setting, the difference $x - y$ is interpreted as the modification introduced by the enhancement process.

3.2 Results

```
[12600/12600] processed
              pipeline  n_files  mean_rms_original  mean_rms_processed  \
... 0             full    12600           0.014834            0.079022
    1  bandpass_wiener    12600           0.014834            0.080942
    2         bandpass    12600           0.014834            0.098301
    3             base    12600           0.014834            0.098603

       mean_centroid_original  mean_centroid_processed  mean_centroid_delta  \
    0             1378.260116               556.700277          -821.559839
    1             1378.260116               518.097220          -860.162896
    2             1378.260116              1060.733952          -317.526164
    3             1378.260116              1574.411631           196.151515

       mean_lsd  median_lsd   std_lsd   mean_snr  median_snr
    0  1.573742    1.553724  0.246791 -13.623142  -13.792725
    1  1.655728    1.624655  0.288879 -13.827862  -14.017040
    2  2.089673    2.072741  0.455315 -15.663027  -15.838380
    3  2.407541    2.397656  0.593237 -15.669033  -15.834191
```

Figure 5: Comparison of filtering pipelines

The **baseline** pipeline, which only performs DC removal and normalization, produced the largest spectral distortion with a mean LSD of 2.41. In addition, this configuration resulted in the highest spectral centroid after processing 1574.41 Hz, indicating that high-frequency components remain largely unchanged.

Adding **speech bandpass filtering** significantly reduced spectral distortion. The mean LSD decreased to 2.09, and the spectral centroid shifted from 1378.26 Hz in the original recordings to 1060.73 Hz after filtering. This indicates that the bandpass filter effectively suppresses both low-frequency and high-frequency noise components.

Further improvements were observed when combining **bandpass filtering** with **Wiener filtering**. In this configuration, the mean LSD decreased to 1.66, demonstrating a stronger reduction of noise while preserving the speech structure. The spectral centroid was further reduced to 518.10 Hz, reflecting stronger attenuation of high-frequency noise.

The best results were obtained with the **full enhancement pipeline**. This configuration achieved the lowest mean LSD of 1.57, indicating the smallest spectral distortion among all tested methods. The spectral centroid after processing was 556.70 Hz, which suggests that high-frequency noise was effectively suppressed while maintaining the core speech band.

A similar trend is observed in the proxy SNR values. The full pipeline achieved the highest SNR among the tested methods (-13.62 dB), indicating the smallest difference between the original and processed signals.

The ablation study demonstrates that each additional filtering stage contributes to improved signal quality. While bandpass filtering alone already reduces spectral distortion, the combination of Wiener

filtering and spectral gating provides the most consistent improvements. The full pipeline therefore represents the most effective preprocessing strategy for the TIMIT dataset in this study.

Variant	**PER**
original	0.10415
base	0.10419
bandpass	0.10664
bandpass_wiener	0.14987
full	0.23881

Table 1: PER for different audio preprocessing pipelines on the TIMIT dataset

The baseline configuration **original** achieved a PER of 0.10415. Applying only basic preprocessing steps such as DC offset removal and RMS normalization - **base pipeline** resulted in a nearly identical performance with a PER of 0.10419. This indicates that simple amplitude normalization does not significantly alter the acoustic characteristics relevant for phoneme recognition.

The **bandpass filtering** pipeline slightly degraded the performance, producing a PER of 0.10664. Although bandpass filtering removes low-frequency drift and high-frequency noise outside the speech band, it also slightly modifies the spectral distribution of the signal. Since the recognition model was trained on the original TIMIT recordings, even small spectral changes may negatively affect recognition accuracy.

A larger degradation was observed when **Wiener filtering** was added to the **bandpass pipeline**. The **bandpass_wiener** configuration increased the PER to 0.14987. Wiener filtering performs adaptive noise suppression, but it may also smooth important phonetic details in the signal. These details, such as transient high-frequency components, are important for distinguishing between certain phonemes.

The largest degradation was observed for the **full enhancement pipeline**. In this configuration the PER increased to 0.23881. Although this pipeline significantly modifies the acoustic signal and may improve perceptual noise suppression, it also alters the statistical distribution of the audio features. As a result, the recognition model, which was trained on the original dataset distribution, performs substantially worse.

Overall, the results show that aggressively improving sound quality does not guarantee an increase in the accuracy of automatic phoneme recognition. While the filtering algorithms may improve certain signal-level properties of the recordings, they can also distort subtle phonetic cues required by the model. Since the acoustic model was trained on the original TIMIT dataset, strong modifications to the input signal lead to a mismatch between training and evaluation data distributions, which ultimately increases the phoneme error rate.